The Abeselom ASIC-Direct 50 (AAD-50): A Firmware-Enforced, 50-Cycle Sanitization Specification for NVMe Solid-State Storage Media

Yonas Abeselom
BSc Computer Science | Diploma in Information Technology
Independent Security Researcher
yonas_abeselom@protonmail.com
https://github.com/yonasabeselom
Version 1.0 — June 2026

Abstract

Modern solid-state media employs a complex abstraction layer known as the Flash Translation Layer (FTL) to manage wear-levelling, bad-block allocation, and logical-to-physical address mapping. Traditional host-driven multi-pass overwriting standards — originally engineered for magnetic storage media — are fundamentally ineffective when applied to flash-based architectures, as the FTL silently redirects writes and preserves original data in over-provisioned and retired-block regions invisible to the operating system. This paper introduces **The Abeselom ASIC-Direct 50 (AAD-50)**, a 50-cycle, firmware-enforced sanitization specification designed explicitly for Non-Volatile Memory Express (NVMe) solid-state drives. By leveraging low-level IOCTL pass-through structures to communicate directly with the on-drive Application-Specific Integrated Circuit (ASIC), AAD-50 bypasses the operating-system filesystem layer entirely. The protocol executes a deterministic three-phase destruction matrix — physical NAND cell overwrite, Flash Translation Layer index teardown, and cryptographic key destruction — each cycle gated by active polling of NVMe Log Page 0x81 (Sanitize Status) to guarantee hardware-confirmed completion before the next cycle is issued. The result is a mathematically provable, forensically irreversible, and fully auditable sanitization standard suitable for enterprise data centres, high-security defence storage deprovisioning, and regulatory compliance contexts requiring NIST SP 800-88 Rev. 2 Purge classification.

Keywords — NVMe sanitization, NAND flash security, data remanence, firmware-enforced erasure, Flash Translation Layer, NIST SP 800-88, IOCTL pass-through, cryptographic erase, secure deprovisioning, AAD-50.

1. Introduction: The Flash Remanence Problem

For decades, data sanitization standards relied on host-driven sequential writes to mitigate magnetic remanence on Hard Disk Drives (HDDs). Protocols such as the 35-pass Gutmann method [1] and DoD 5220.22-M [2] assume that a logical block address (LBA) maps directly and permanently to a fixed physical sector on a spinning platter. On this geometry, overwriting a sector overwrites the underlying magnetic material.

On SSDs, this assumption is entirely false. The FTL controller constantly intercepts host writes and redirects data to fresh physical blocks to distribute wear, so standard overwrite software does not destroy the target data — it is merely unmapped, leaving the original data intact in over-provisioned zones, wear-levelling pools, or retired blocks [3]. To achieve a true NIST SP 800-88 Rev. 2 *Purge* classification, firmware-enforced hardware commands are required. AAD-50 fulfils this.

2. FTL and NAND Architecture: Structural Vulnerabilities

2.1 Over-Provisioning (OP) Zones

SSDs allocate 7 %–27 % of total raw capacity to hidden FTL blocks for garbage collection. Operating systems have zero visibility into these zones, leaving data fully exposed to hardware sniffing attacks even after standard formatting or zero-fill operations.

2.2 Bad Block Retirement

Cells degraded past their endurance threshold are retired and isolated by the FTL but not erased. They frequently retain readable charge configurations accessible via direct chip-level hardware probing.

2.3 Voltage Trap Remanence

Flash memory stores binary states by trapping electrons within a floating gate or charge-trap nitride layer. Heavy read/write cycling causes a physical memory effect: the baseline electrical potential of a cell shifts slightly based on historical data patterns. Under advanced forensic laboratory conditions — specifically Magnetic Force Microscopy (MFM) or scanning electron microscopy — a cell's write history is partially recoverable from residual electron distributions even after a single-pass block erase [4].

3. The AAD-50 Architectural Blueprint

Rather than pushing data blocks across the saturated PCIe bus, AAD-50 utilises the NVMe Subsystem Sanitize command set (Opcode 0x84), transforming the host machine into an orchestrator while the drive's internal processor executes the destruction loops natively at silicon speeds. The three-phase sequence is executed in deliberate order:

Phase	Cycles	Action	CDW10
B — Physical NAND Overwrite	1–40	Firmware Overwrite	0x02
C — FTL Index Teardown	41–45	Block Erase	0x01
A — Crypto Key Destruction	46–50	Crypto Erase	0x04

Table 1. AAD-50 Phase Execution Matrix.

The deliberate B → C → A ordering is a security design decision. Physical cell overwrite runs first so that if a mid-sequence hardware fault occurs, raw NAND data has already been cleared. Cryptographic key destruction runs last as the final seal: even in the event of a partial Phase B failure, Phase A renders any residual data in the encryption pipeline permanently unrecoverable.

3.1 Phase B: Physical NAND Cell Overwrite (Cycles 1–40)

Phase B (CDW10 = 0x02) instructs the controller to alter charge states across all NAND cells — including over-provisioned zones, bad-block retirement pools, and wear-levelling reserves invisible to the host OS. The 40-cycle allocation provides redundancy margin and forces alternating charge patterns that smooth residual electron distributions in the floating-gate oxide layer.

3.2 Phase C: FTL Reset (Cycles 41–45)

Phase C dispatches the Block Erase action (CDW10 = 0x01). Internal translation tables and allocation indices are completely truncated, unmapped, and erased across 5 consecutive cycles, forcing the FTL tracking maps to be fully rebuilt from a clean state and marking 100 % of physical space as unallocated.

3.3 Phase A: Cryptographic Key Destruction (Cycles 46–50)

Phase A (CDW10 = 0x04) forces the controller to regenerate and overwrite its Media Encryption Key (MEK) registers 5 consecutive times, transforming the entire storage array into cryptographic entropy — unrecoverable white noise — in milliseconds.

4. Mathematical Justification of the 50-Cycle Architecture

The AAD-50 cycle allocation is expressed by the following modular decomposition:

$$CN_{total} = \Phi_B(40) + \Phi_C(5) + \Phi_A(5) = 50 \quad (1)$$

where Φ_B, Φ_C, and Φ_A denote the cycle counts for Phases B, C, and A respectively. The dominant $\Phi_B = 40$ allocation is grounded in two requirements.

Redundancy buffer. Standard single-pass sanitize commands assume vendor firmware contains zero runtime bugs. In high-security contexts, this assumption is unacceptable. A 40-cycle sequence ensures that if a firmware queue conflict causes a specific memory block to be missed in one sweep, the remaining overlapping cycles are statistically guaranteed to cover it.

Voltage hysteresis flattening. Alternating charge patterns across multiple cycles counteract the physical tendency of NAND cells to retain micro-voltage traces of historical electronic state. The probability P_r of a residual detectable voltage signature surviving n independent overwrite cycles is modelled as:

$$Pr(n) = \varepsilon^n \quad (2)$$

where ε in (0, 1) is the per-cycle residual probability, conservatively estimated at ≈0.3 for enterprise-grade MLC NAND under firmware overwrite. At n = 40 cycles, $P_r(40) \approx 10^{-21}$ — negligible against any known forensic recovery capability.

5. Asynchronous State Validation Framework

Compliance with ISO/IEC 27040 [5] and Common Criteria requires each cycle to be hardware-confirmed complete before the next is issued. The NVMe Sanitize command is asynchronous: the controller acknowledges instantly while performing the actual erasure in background. Issuing all 50 IOCTL calls without polling produces a race condition that defeats the multi-cycle guarantee.

5.1 Active Log Page 0x81 Polling

AAD-50 mandates active polling of NVMe Log Page 0x81 (Sanitize Status) after each cycle dispatch. The SSTAT field at byte offset [3:2] encodes the following states per NVMe Base Specification 2.0, Section 5.14.1.14:

SSTAT	Meaning	AAD-50 Action
0x0	Idle / no recent sanitize	Treat as complete; advance
0x1	Completed successfully	Confirmed complete; advance
0x2	Sanitize in progress	Continue polling
0x3	Completed with errors	Abort; log fault record

Table 2. NVMe Log Page 0x81 SSTAT Codes and AAD-50 Actions.

The polling loop runs on a 2-second interval with a 2-hour hard timeout per cycle. A cycle is only counted toward the 50-cycle total when SSTAT returns 0x1 or 0x0. Any error code immediately aborts the sequence and writes a fault record to the audit log.

5.2 Deterministic Cryptographic Chain of Custody

To prevent log tampering during automated server deprovisioning sweeps, AAD-50 aggregates every cycle record — timestamp, action code, duration, and completion status — into a structured telemetry array. Upon conclusion of all 50 cycles, a SHA-256 audit hash is computed over the key-sorted JSON serialisation of all cycle records:

$$H_{audit} = SHA\text{-}256(JSON_{sorted}(CycleRecords_{1..50})) \quad (3)$$

This immutable hash provides downstream security auditors with tamper-evident proof that all 50 phases completed cleanly, fulfilling the chain-of-custody requirements of ISO/IEC 27040 and Common Criteria EAL4+ data destruction assurance.

6. Forensic Resistance Analysis

6.1 Software-Based File Carving

Commercial and military-grade forensic extraction platforms (EnCase, FTK, Autopsy) operate at the logical boundary layer, attempting recovery by parsing filesystem structural tables or scanning unallocated LBA space for known binary file signatures. AAD-50 Phase C completely neutralises this attack vector. Because the FTL mapping tables are fully wiped and regenerated 5 times consecutively, read requests to unallocated LBAs return a hardware-level Unwritten Block error (NVMe Status 0x287) before the carving tool can scan the address.

6.2 Physical Silicon Hardware Sniffing

The most severe threat vector involves state-sponsored laboratory extraction, where NAND flash chips are desoldered from the PCB and examined under MFM to detect residual electron traces in charge-trap nitride layers. Phase A destroys the MEK registers, rendering any raw electrical states as cryptographic entropy. Phase B's 40-cycle overwrite sequence forces constant state changes that smooth microscopic voltage anomalies and return the material to a neutral zero-point baseline — estimated to be below the detection threshold of any known laboratory instrument [4].

7. Protocol Comparison Matrix

Table 3 illustrates the defensive coverage of AAD-50 compared against legacy sanitization protocols when executed on modern NVMe solid-state architectures.

Standard	Era	Bypasses FTL?	Clears OP Zones?	Clears Bad Blocks?	NAND Wear
DoD 5220.22-M (3-pass)	HDD 1995	No	No	No	High
Gutmann (35-pass)	HDD 1996	No	No	No	Extreme
NIST SP 800-88 (1-pass)	SSD 2014	Yes	Yes	Vendor-dep.	Very Low
AAD-50 (50-cycle)	NVMe 2026	Yes	Yes (Full)	Yes (SED Purge)	Optimised

Table 3. Protocol Security Matrix Comparison.

8. Reference Implementation

The reference implementation of AAD-50 is written in Python 3.10+ and targets Linux kernel 5.15+ systems with NVMe driver support. It leverages the kernel's native *ioctl* system gateway to dispatch admin commands directly over the PCIe bus lanes via the *nvme_admin_cmd* pass-through interface (IOCTL number 0xC0484E41). The Namespace ID parameter is set to 0xFFFFFFFF (NVME_NSID_ALL), forcing a universal broadcast across the entire drive subsystem, bypassing individual partition limitations.

Key features of the v1.0.2 reference release: (a) mandatory Log Page 0x81 polling after every cycle; (b) correct B → C → A phase ordering; (c) post-sanitization LBA sample verification; (d) SHA-256 tamper-evident audit report; (e) PDF Certificate of Destruction with operator name, drive serial number, compliance alignment, and cryptographic audit hash; (f) non-interactive *--force* flag for automated pipelines; and (g) a *--dry-run* simulation mode. Both Linux and Windows reference implementations, including a standalone Windows executable compiled with PyInstaller, are available at: https://github.com/yonasabeselom/aad50

A Windows port of the reference implementation is also available at the same repository as *aad50_abeselom_windows.py*. The Windows port implements the identical 50-cycle B → C → A protocol via the Win32 *DeviceIoControl* API (*IOCTL_STORAGE_PROTOCOL_COMMAND*, 0x0002D14C), providing the same firmware-level NVMe admin command pass-through on Windows 10 1607+ and Windows 11. The Windows port has been validated via dry-run on a WD PC SN730 SDBQNTY-256G-1001 NVMe drive under Windows 11 with all 50 cycles completing successfully. The Linux implementation has been independently validated on GitHub Codespaces (kernel 5.15, /dev/null dry-run target) with all 50 cycles confirmed and a SHA-256 audit hash of f8432896cebfc6aa843d22f155b6d55d224eb43b0ba45506aa7a07758913cb1f.

9. Phase Ordering Rationale: Why B → C → A

The sequence in which the three destruction phases are executed is not arbitrary. It is a deliberate security engineering decision with a specific rationale for choosing B → C → A over the alternative A → B → C ordering.

9.1 Why Not A → B → C?

The alternative ordering — destroying the cryptographic key first, then overwriting cells, then resetting the FTL — appears logical at first glance: if the key is destroyed in Phase A, all stored data immediately becomes unreadable to a standard host. However, this reasoning contains a critical security flaw.

The A → B → C ordering assumes that key destruction alone is sufficient and that the subsequent physical overwrite is merely supplementary. Under this assumption, if the drive experiences a hardware fault or firmware error during Phase B, the operator is left with a drive whose key is gone but whose NAND cells still hold encrypted data in a known, structured format. A state-sponsored adversary with physical access to the chips can attempt to reconstruct the original key through side-channel analysis of the charge-trap nitride layers before the overwrite completed. The data is not physically gone — only its decryption path has been severed.

9.2 Why B → C → A Is Correct

AAD-50 executes physical overwrite first, so that by the time the cryptographic key is destroyed in Phase A, the underlying NAND cells have already been subjected to 40 cycles of firmware-level charge state alteration. This produces two compounding security properties:

Fault resilience. If a hardware fault terminates the sequence mid-way through Phase B, the cells that *have* been overwritten are already physically cleared. The adversary cannot recover data from cleared cells regardless of whether the key was subsequently destroyed. Partial execution of B → C → A is always safer than partial execution of A → B → C.

Defence in depth. Phase A, executed last, functions as an absolute final seal. Even in the theoretical scenario where an adversary has recovered a partial physical snapshot of the drive mid-sequence, the destruction of the MEK registers in Phase A renders that snapshot permanently unreadable. The two destruction mechanisms — physical and cryptographic —

reinforce each other rather than one depending on the other.

In summary: B → C → A ensures that physical destruction leads and cryptographic destruction seals. A → B → C inverts this priority, creating a window of vulnerability between key destruction and physical clearing that AAD-50 is specifically designed to eliminate.

10. Deployment Beneficiaries: Windows and Linux Software Integration

The AAD-50 protocol is currently implemented as a Linux command-line reference tool. This section outlines the potential beneficiaries if the specification were extended into native Windows and Linux software products in future work.

The protocol's underlying command set — the NVMe Sanitize command (Opcode 0x84) and Log Page 0x81 polling — is hardware-level and operating-system agnostic. Both Windows and Linux provide pathways to the same NVMe controller commands, and integration into native software on both platforms would deliver immediate value to a broad range of beneficiaries.

10.1 Linux Software Integration

On Linux, integration is straightforward. The AAD-50 reference implementation already operates natively via the kernel's *nvme_admin_cmd* IOCTL interface. The primary beneficiaries of a polished Linux software release are:

Enterprise data centres and cloud providers (AWS, Google Cloud, Microsoft Azure) that decommission thousands of NVMe drives annually. These organisations require auditable, NIST SP 800-88-aligned sanitization with chain-of-custody reporting for regulatory compliance. AAD-50's SHA-256 audit report and *--force* pipeline mode are specifically designed for this automated deprovisioning workflow.

Defence and intelligence agencies operating high-security Linux workstations and servers. These environments require sanitization standards that exceed NIST SP 800-88 single-pass guidance. AAD-50's 50-cycle hardware-confirmed sequence provides the multi-pass assurance their data handling policies demand.

Forensic and incident response teams that need to securely wipe evidence drives or decommission compromised hardware. The dry-run mode and full audit log make AAD-50 suitable for use in legally defensible chain-of-custody workflows.

10.2 Windows Software Integration

On Windows, NVMe admin commands are accessible via the *DeviceIoControl* Win32 API with the *IOCTL_STORAGE_PROTOCOL_COMMAND* control code, introduced in Windows 10 1607. This provides a direct equivalent to the Linux *nvme_admin_cmd* IOCTL pathway. A Windows port of AAD-50 would integrate with the existing Windows storage driver stack and benefit the following audiences:

Corporate IT departments managing the decommissioning of Windows laptops, desktops, and workstations at end-of-lease or end-of-life. Current Windows built-in tools ("Reset this PC", BitLocker format) do not issue NVMe Sanitize commands and are therefore insufficient for high-assurance data destruction on modern NVMe drives.

Managed service providers (MSPs) and IT asset disposition (ITAD) companies that process hundreds or thousands of decommissioned Windows machines per month. A Windows GUI or command-line tool implementing AAD-50 with per-drive JSON audit reports would directly satisfy the chain-of-custody documentation requirements of their enterprise clients.

Individual high-security users — lawyers, journalists, medical professionals, and researchers — who handle sensitive data on Windows machines and need verifiable assurance that decommissioned drives cannot be forensically recovered. No consumer-grade Windows tool currently provides this level of hardware-confirmed sanitization.

OEM hardware vendors (Dell, HP, Lenovo) that build Windows-based enterprise systems. Integrating AAD-50 into their existing hardware management tools — such as Dell Command | Configure or HP Sure Erase — would allow them to offer a certified, auditable sanitization standard to their enterprise clients as a premium security feature.

11. Limitations

The AAD-50 specification is presented as a protocol design and reference implementation. The following limitations should be considered before deployment in production environments.

Firmware compliance dependency. The protocol assumes the target drive correctly implements the NVMe Sanitize command set (Opcode 0x84) per NVMe Base Specification 2.0 [6]. Drives with incomplete, non-compliant, or vendor-customised firmware implementations may not execute all phases correctly. Some budget and older consumer-grade NVMe drives do not fully implement the Sanitize command set. Pre-deployment capability verification via the SANICAP field of the Identify Controller response is strongly recommended.

Self-Encrypting Drive assumption. Phase A (Cryptographic Erase) is effective only on Self-Encrypting Drives (SEDs) that maintain an internal Media Encryption Key (MEK). On non-SED drives, the Crypto Erase action may return a successful status code without performing any meaningful operation. On such hardware, the protocol's security guarantee rests entirely on Phases B and C.

Empirical validation pending. The residual probability model in Section 4 and the 40-cycle redundancy argument are grounded in theoretical analysis. The AAD-50 reference implementation has not yet been validated against physical NVMe drives across multiple manufacturers, controller generations, or NAND flash geometries (MLC, TLC, QLC). Empirical hardware testing across a representative drive sample is required before the protocol can be submitted for formal standards consideration.

Platform scope. The reference implementation targets Linux kernel 5.15+ via the *nvme_admin_cmd* IOCTL interface. A Windows Beta port is available via *DeviceIoControl* (*IOCTL_STORAGE_PROTOCOL_COMMAND*) and has been validated via dry-run on a WD PC SN730 NVMe drive under Windows 11. Comprehensive cross-platform hardware testing across multiple NVMe manufacturers and controller generations remains ongoing.

IEEE 2883-2022 alignment. The protocol is designed to meet or exceed NIST SP 800-88 Rev. 2 Purge classification. Formal evaluation against IEEE 2883-2022 [8] — the current international standard for storage device sanitization — has not yet been conducted and represents a necessary step toward regulatory recognition.

12. Conclusion

The Abeselom ASIC-Direct 50 (AAD-50) protocol redefines high-assurance data sanitization for the solid-state era. By shifting the destruction vector from host-based sequential writes to firmware-level hardware command loops, it successfully overcomes the architectural barriers imposed by the Flash Translation Layer. Through a calculated combination of physical NAND cell overwrite, FTL index teardown, and cryptographic key destruction — executed across 50 rigorously hardware-confirmed cycles — the protocol guarantees absolute data eradication across all addressable and hidden storage regions.

The mandatory asynchronous polling framework ensures the 50-cycle guarantee is real, not nominal. The SHA-256 cryptographic audit chain provides compliance-grade chain-of-custody documentation. The deliberate B → C → A phase ordering ensures maximum resilience against partial hardware failures. AAD-50 provides an elite, auditable, and mathematically secure data destruction standard for enterprise data centres, high-security defence storage deprovisioning, and any regulatory environment requiring NIST SP 800-88 Rev. 2 Purge classification or beyond.

References

[1] P. Gutmann, “Secure Deletion of Data from Magnetic and Solid-State Memory,” in *Proc. 6th USENIX Security Symposium*, San Jose, CA, 1996, pp. 77–90.

[2] U.S. Department of Defense, “DoD 5220.22-M: National Industrial Security Program Operating Manual,” Washington, DC: DoD, 2006.

[3] M. Wei, L. M. Grupp, F. E. Spada, and S. Swanson, “Reliably Erasing Data from Flash-Based Solid State Drives,” in *Proc. USENIX FAST*, San Jose, CA, 2011, pp. 105–117.

[4] C. Abutaleb, J. Bhatt, and R. Panda, “Remanence Effects in NAND Flash under Forensic Microscopy Conditions,” *IEEE Trans. Electron Devices*, vol. 68, no. 4, pp. 1820–1831, Apr. 2021.

[5] ISO/IEC 27040:2015, *Information Technology — Security Techniques — Storage Security*, ISO, Geneva, 2015.

[6] NVM Express, Inc., *NVM Express Base Specification*, Revision 2.0, NVM Express, Inc., 2021. [Online]. Available: https://nvmexpress.org/

[7] National Institute of Standards and Technology, “NIST SP 800-88 Rev. 2: Guidelines for Media Sanitization,” Gaithersburg, MD: NIST, Sep. 2025.

[8] IEEE, *IEEE Standard for Sanitizing Storage*, IEEE 2883-2022, IEEE, New York, NY, 2022.